

Disclaimer: Dieses OER-Dokument stammt aus dem Projekt [proTirone](#), das über GitHub [freie Unterrichtsmaterialien](#) samt [Quellcode](#) offeriert. proTirone-Materialien werden unter den Bedingungen der [CC-BY-4.0-Lizenz](#) so angeboten, wie sie sind, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).

LF09:12:Netzanalyse:

[→ ZP:Sheet:2]

1. Überblick

Tool(LNX)	Information(en) zu	Tool(W11)
<code>arp</code>	aktive MAC-Adressen in Broadcastdomäne (+ Zusatzinfos)	<code>arp -a -v</code>
<code>ip neighbor</code>	(aktive) MAC- und IP-Adressen in Broadcastdomäne	<code>netsh interface ipv4 show neighbors</code>
<code>ifconfig</code>	Netzmaske, IP- und Broadcastadresse	<code>ipconfig /all</code>
<code>ip address</code>	IP-Adresse in CIDR-Notation, Broadcast- und MAC-Adresse	<code>netsh interface ipv4 show addresses</code>
<code>ip route</code>	Netzadresse in CIDR-Notation, IP-Adresse und Gatewayadresse	<code>route print</code>
<code>ping</code>	Erreichbarkeit eines 'Rechners' auf IP-Ebene	<code>ping</code>
<code>tracert</code>	Hoppings auf dem Weg zum anderen Rechner	<code>tracert</code>
<code>nslookup</code>	IP-Adresse zu Domainnamen (u.umgekehrt)	<code>nslookup</code>
<code>netstat -r</code>	Routingtabelle	<code>netstat -r</code>
<code>netstat -ano</code>	alle lauschenden und etablierte TCP- u. UDP-Ports	<code>netstat -ano</code>

2. Demofahrplan:

1. mit `arp` die via ARP-Cache aktuell bekannten Rechner ermitteln
2. mit `ip neighbor` feinere Informationen dazu abfragen
3. mit `ping` fehlende Rechner kontaktieren
4. \1. und/oder \2. wiederholen und die Cache-Auffrischung verifizieren
5. mit `ip address` eigene IP-Adresse in CIDR-Notation, Broadcast- und MAC-Adresse ermitteln
6. mit `ip route` Netzadresse in CIDR-Notation, IP-Adresse und Gatewayadresse abfragen
7. mit `netstat -r` verifizieren
8. mit `ping 8.8.8.8` Erreichbarkeit Google-DNS-Server abfragen
9. mit `traceroute 8.8.8.8` Hoppings durch private-Subnetze und durchs Internet hindurch abfragen
10. mit `nslookup dns.google.com` IP-Adresse vom Google-DNS-Server erfragen
11. mit `nslookup 8.8.8.8` zugehörigen Domainname erfragen
12. mit `netstat -ano | grep tcp` lauschenden und etablierte TCP-Ports abfragen (:80)

3. Einzelinformationen

3.1 arp (unter W11: `arp -a`)

“[...] manipulates or displays the kernel’s IPv4 network neighbour cache.” [→ `man(arp)`]

- Informiert über gerade im Netz agierende Kommunikationspartner.
- Enthält die eigene MAC-Adresse.

Beispiel:

	Address	HWtype	HWaddress	Flags	Mask	Iface
1	.	ether	b0:5c:da:f7:db:31	C		wlp113s0f0
2	speedport.ip	ether	c4:e5:32:15:a6:bc	C		wlp113s0f0

3.2 ifconfig (unter W11: `ipconfig /all`)

“[...] is used to configure [or display] the kernel-resident network interfaces” [→ `man(ifconfig)`]

- `ifconfig` (pure): “displays the status of the currently active interfaces”
- `ifconfig wlp113s0f0` (mit Interfacename): “displays the status of the given interface only”
- `ifconfig -a`: “displays the status of all interfaces, even those that are down”

Status enthält jeweils:

- IPv4-Adresse + IPv4-Netzmaske

- alle an das Interface gebundene IPv6-Adressen mit Prefixlänge
- MAC-Adresse

Beispiel:

```
1 inet 192.168.2.102 netmask 255.255.255.0 broadcast 192.168.2.255
2 inet6 2003:c0:bf04:69cc:d9fe:6524:22a4:c939 prefixlen 64 scopeid 0x0<
  global>
3 inet6 2003:c0:bf04:69cc:bc6c:6eb:2009:c6e8 prefixlen 64 scopeid 0x0<
  global>
4 inet6 fe80::3b8:77e6:12cc:b8b3 prefixlen 64 scopeid 0x20<link>
5 ether a0:d3:65:d3:60:ee (Ethernet)
```

3.3 ip (unter W11: netsh)

“[...] show[s] / manipulate[s] routing, network devices, interfaces and tunnels [→ man(ip)]

- **ip addr** “shows addresses assigned to all network interfaces” (= IPv4- und IPv6-Adressen in CIDR-Notation)
- **ip neigh** “shows the current neighbour table in kernel.” (= Infos über BCD-Partner)
- **ip route** “show table routes” (= (Default) Gateway(s))

ist moderner als ipconfig, liefert feinere Aussagen.

Nette Einzelanwendungen:

- **ip address show** (= IPv4- und IPv6-Adressen in CIDR-Notation)
- **ip -4 address show** (= IPv4-Adressen in CIDR-Notation)
- **ip -6 address show** (= IPv6-Adressen in CIDR-Notation)
- **ip -0 address show** (= MAC-Adressen)
- **ip address show** liefert auch reale und gewünschte Lebenszeit der IP-Adresse (valid_lft preferred_lft)
- **ip -ts monitor label** zeigt Nachrichten des Netlink Bus des Kernels (“was ist gerade wirklich los”)
- **ip -6 -ts monitor neigh** zeigt Nachrichten des Neighbour Caches an.
- **ip neighbor show** zeigt - wie arp - den Cache des des „Address Resolution Protocol“ (ARP) an
- **ip -6 neighbor show** zeigt das Äquivalent zum IPv4-ARP-Cache in IPv6 an, den sogenannten „Neighbor Cache“
- **ip route show** zeigt die ipv4-Routingtabelle an
- **ip -6 route show** zeigt die ipv6-Routingtabelle an
- **ip -stats -human link show** zeigt eine Paketstatistik an

3.4 netsh nur unter Windows 11

- steht für *Network Shell* [→ <https://learn.microsoft.com/de-de/windows-server/administration/windows-commands/netsh>]
- wird von einer Shell – z.B. pwsh – aus aufgerufen
- ist selbst eine Shell, die Netzwerkbefehle entgegennimmt und ausführt
- arbeitet mit ineinander verschachtelten Kontexten gefolgt von einem Befehl und dessen Parameter

Beispiele:

- `netsh interface ipv4 show addresses` (Befehl: *show* Parameter: *addresses*)
- `netsh interface ipv4 show interfaces interface=17` (Befehl: *show* Parameter: *interfaces interface=17*)
- `netsh interface ipv6 show addresses` (Befehl: *show* Parameter: *addresses*)

4 ping unter LNX/W11

“send[s] ICMP ECHO_REQUEST to network hosts” [→ `man(ping)`]

- `ping 8.8.8.8` sendet Echorequest zu einem Googleserver. Liefert Dauer bis zur Rückkehr.
- gut, um das prinzipielle Funktionieren der Netzkonfiguration zu verifizieren

3.5 traceroute (unter W11: `tracert`)

“print[s] the route packets trace to network host” [→ `man(traceroute)`]

- ermittelt die Hoppings bis zum Zielrechner.
- erlaubt Einblick in Netzwerkstruktur

Beispiel:

```
1 1 speedport.ip (192.168.2.1) 6.802 ms 6.740 ms 6.727 ms
2 2 p3e9bf608.dip0.t-ipconnect.de (62.155.246.8) 12.008 ms 11.997 ms
   11.986 ms
3 3 f-ed11-i.F.DE.NET.DTAG.DE (62.154.17.218) 27.403 ms
4   f-ed11-i.F.DE.NET.DTAG.DE (217.5.109.22) 14.145 ms
5   f-ed11-i.F.DE.NET.DTAG.DE (62.154.17.254) 14.063 ms
6 4 * * *
7 5 * * *
8 6 dns.google (8.8.8.8) 13.242 ms 7.956 ms 8.971 ms
```

3.6 nslookup unter LNX/W11

“quer[ies] Internet name servers interactively” [→ man(nslookup)]

- erhält Domänenname, liefert IP-Adresse
- erhält IP-Adresse, liefert Domännennamen (reverse ns-lookup)

Beispiel:

```
1 nslookup karsten-reincke.de
2
3 Non-authoritative answer:
4 Name:    karsten-reincke.de
5 Address: 81.169.145.160
6 Name:    karsten-reincke.de
7 Address: 2a01:238:20a:202:1160::
```

```
1 nslookup 8.8.8.8
2 8.8.8.8.in-addr.arpa    name = dns.google
3
4 nslookup 81.169.145.160
5 160.145.169.81.in-addr.arpa name = wa0.rzone.de
```

Nette Einzelanwendungen:

`nslookup -query=A google.de` liefert Ihnen die IPv4-Adresse `nslookup -query=AAAA google.de` liefert Ihnen die IPv6-Adresse

3.7 netstat unter LNX/W11

“[...] print[s] network connections, routing tables, interface statistics [...]” [→ man(netstat)]

liefert

- `netstat -r` liefert Routingtabelle
- `netstat -i` zeigt Infos zu allen Interfaces an
- `netstat -s` zeigt Statistiken für Übertragungsprotokolle

Beispiel:

```
1 netstat -r
2 Kernel IP routing table
3 Destination Gateway Genmask Flags MSS Window irtt Iface
4 default speedport.ip 0.0.0.0 UG 0 0 0
   wlp113s0f0
5 192.168.2.0 0.0.0.0 255.255.255.0 U 0 0 0
   wlp113s0f0
6
```

Nette Einzelanwendungen:

- `netstat -ano | grep tcp` liefert die lauschenden und etablierten TCP-Socket-Verbindungen
- `netstat -ano | grep udp` liefert die lauschenden und etablierten UDP-Socket-Verbindungen

Ähnlich: `lsof -i -n`

ÜBUNG LF09:12:Netanalysis:01

- ☐ Machen Sie sich zuerst mit allen Tools unter Linux vertraut. Nutzen Sie dazu das Schulnetz.
- ☐ Machen Sie sich danach mit den Windows Äquivalenten vertraut. Nutzen Sie dazu das Schulnetz.
- ☐ Finden Sie so viel wie möglich über den Aufbau des Schulnetzes heraus.
- ☐ Dokumentieren Sie die Struktur des Schulnetzes, so weit Ihnen erkennbar.

Solution: [→ **ZP:Sheet:3**]

• Netz:

- aus `ifconfig` WLAN-Schnittstellen-ID ablesen.
- aus `ifconfig $WLID` eigene IP-Adresse, Netz-Maske, Netz-Adresse, BCD-Adresse ablesen.
- aus `ip route show` Gateway-Adresse Netz-Adresse mit Präfixlänge ablesen.
- mit `ipaddress.ip_address(...)` Gateway-Definitionsmethode feststellen (NT+1 oder BC-1?)
- **Ergebnis:** [Stand 12.2.26:] 10.75.64.0/19
 - * 10.75 = wohl ein aus privatem Klasse-A-Netz (/8) abgespaltenes Klasse B-Netz (/16)
 - * 10.75.64 = aus dem Klasse-B-Netz abgespaltenes /19-Subnetz
 - * weitere aus 10.75/16 abspaltbare /19-Subnetze:
 - 10.75.00 - 10.75.31.255 [*hätte 10.75.20.1 als Mittendrin-Gateway. Realistisch?*]
 - 10.75.32 - 10.75.63.255
 - **10.75.64 - 10.75.95.255** [existiert]
 - 10.75.96 - 10.75.127.255 [*hätte 10.75.110.1 als Mittendrin-Gateway. Realistisch?*]
 - 10.75.128 - 10.75.159.255
 - 10.75.160 - 10.75.191.255
 - 10.75.192 - 10.75.223.255

- 10.75.224 - 10.75.255.255

- * Segmentierung kann aber auch ganz anders sein

- **Netzumgebung:**

- mit `tracert` 8.8.8.8 Weg zu Google durch Schulnetze ermitteln:
- Nbook → WLAN-GW:10.75.95.254 → ZWGW:10.75.20.1 → EDGEGW:10.75.110.1 → Internet
- `nslookup www.gs-ldk.de` → IP-Adresse 'Schulwebserver': 178.254.11.61
- `nslookup gs-ldk.de` → IP-Adresse 'Domainserver': 178.254.11.61 (Hypothese: ders. also auch kein ssh.gs-ldk.de)
- `nslookup www.gs-ldk.eu` → IP-Adresse 'Schulserver': 80.144.108.63
- Wege zu Servern:
 - * `tracert 178.254.11.61`: von außen kein Ergebnis, letzte angezeigte IP: 185.195.100.22
 - * `tracert 80.144.108.63`: von außen kein Ergebnis, letzte angezeigte IP: 87.137.244.137
- mit whois-Dienst und IP-Geolocation-Diensten
 - * wie
 - <https://www.heise.de/netze/tools/whois/>
 - <https://whois.ripe.net>
 - <https://www.bennettrichter.de/tools/ip/>
 - <https://nordvpn.com/de/ip-lookup/>
 - * Ort und Anbieter feststellen:
 - 178.254.11.61 ISP: 1blu GmbH, OWNER: EVANZO, ORT: North Rhine-Westphalia/Eschweiler
 - 185.195.100.22 ISP/OWNER: EVANZO, ORT: unknown
 - 80.144.108.63 ISP: Deutsche Telekom AG, OWNER: Deutsche Telekom AG, Internet service provider, ORT: Hessen/Greifenstein
 - 87.137.244.137 ISP: Deutsche Telekom AG, OWNER: Deutsche Telekom AG, Internet service provider ORT: unknown
 - * zusammenfassen:
 - Web-Präsenz der Schule wird von einer Firma EVANZO gehostet
 - Anbindung der Schule über Provider 'Deutsche Telekom'

- **Schulnetzstruktur:** Hypothese: 3 Netze bis Außenpunkt des Schulnetzes spricht für Struktur

- DMZ: Gateway: 10.75.110.1
 - * wenn `ping 10.75.110.0` kein Ergebnis und
 - * wenn `ping 10.75.110.255` => *ping: Do you want to ping broadcast?*

- * dann 10.75.110.0/24 Netz
 - Verwaltungsnetz: Gateway: 10.75.20.1
 - * wenn **ping** 10.75.20.0 kein Ergebnis und
 - * wenn **ping** 10.75.20.255 => *ping: Do you want to ping broadcast?*
 - * dann 10.75.20.0/24 Netz
 - Wlan = Unterrichtsnetz
 - Office-Lan für Fachzimmer und Sekretariat (Hypothese!)
 - Binnen-Server-Lan
-

ÜBUNG LF09:12:Netanalysis:02

Strukturieren und erstellen Sie ein JSON-File, dem Sie entnehmen können,

- ☐ welche sieben zentralen Netzanalysetools wir haben,
 - ☐ wie man sie benutzt und
 - ☐ was man von Ihnen erfährt.
 - ☐ zu welchen Zwecken man mit welchen Netzanalysetools benutzen kann.
-